WELT 5 · DOPPELSTUNDEN 31, 32 UND 33 (KÜR) · LEVEL 5.1 BIS 5.3

W5·1 – Der Wald: Binärbäume

Datei: `AQ_W5_L5_Wald_Geruest.txt` (Klassen Baum, Rekursiv, Akinator: Level 5.1 bis 5.3) · Grafik: `AQ_W5_Baumzeichner_Geruest.txt` · Im Lab (Algo Quest Lab): Level 5.1 bis 5.3 · Teil 1 in DS 31, Teil 2 in DS 32, Teil 3 in DS 33 (Kür)

Niveau (KC 2017): nur Leistungskurs (eA) · Teil 3 (Entscheidungsbaum und Level-Order, Kür) geht über das Kerncurriculum hinaus

Hinter dem Hafen beginnt der **Wald**. Kein Weg führt geradeaus: An jeder Weggabelung teilt sich der Pfad in einen linken und einen rechten Weg, und jeder dieser Wege gabelt sich wieder – bis er an einer Lichtung endet. Wer sich hier nicht verirren will, braucht eine Karte, die genau diese Struktur hat: einen **Binärbaum**.

Für euer Dungeon-Spiel nutzt ihr Bäume gleich mehrfach: als **Akinator-Monsterquiz**, das Monster errät und dabei dazulernt, und (auf dem nächsten Arbeitsblatt) als **Item-Datenbank**, in der jedes Item blitzschnell gefunden wird. Im Abitur kommen Binärbäume in fast jedem Jahrgang vor – als Baum, den du aus Code zeichnest, als Traversierung, als Termbaum und als **rekursive Methode**, die du von Hand schreibst.

Teil 1 – Bäume lesen und bauen (DS 31)

Infoteil 1: Die Sprache der Bäume

Rekursive Definition: Ein Binärbaum ist entweder **leer** oder er besteht aus einer **Wurzel** mit einem Inhalt und zwei Binärbäumen – dem **linken** und dem **rechten Teilbaum**. Jede rekursive Baummethode folgt genau dieser Definition.

Begriff	Bedeutung
Knoten	Element des Baums mit einem Inhalt (auch: Wert, Item)
Wurzel	der oberste Knoten; er hat keinen Elternknoten
Kind / Elternknoten	die Wurzeln der beiden Teilbäume eines Knotens sind seine Kinder; er ist ihr Elternknoten
Blatt	Knoten, dessen beide Teilbäume leer sind (keine Kinder)
innerer Knoten	Knoten mit mindestens einem Kind
Teilbaum	ein Knoten mit allen seinen Nachfahren – selbst wieder ein Binärbaum
Ebene	Abstand zur Wurzel: **Wurzel = Ebene 0**, ihre Kinder Ebene 1 usw.
Höhe	**leerer Baum: 0, Blatt: 1**, sonst 1 + größere Höhe der beiden Teilbäume (= Anzahl der Ebenen)

Achtung: Manche Bücher und Videos zählen die Höhe in Kanten (Blatt = 0). Wir zählen wie oben. Gibt eine Abituraufgabe eine eigene Definition vor (z. B. „Tiefe“), gilt diese.

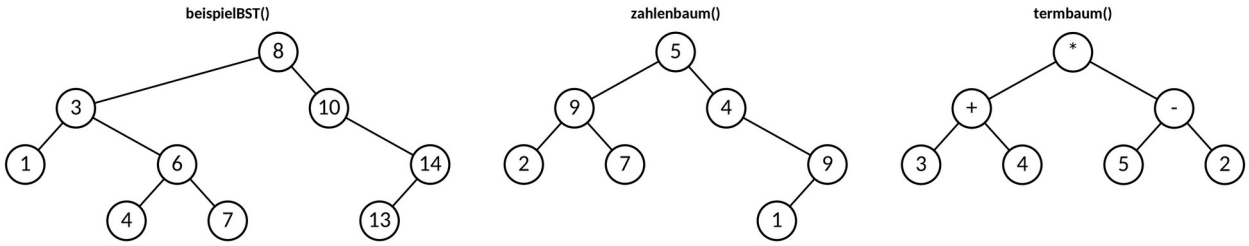


Abbildung 1: Die Testbäume der Baumschule – sie begegnen dir in allen Levels, Karten und im Boss-Kampf

Aufgabe 1: Die alte Eiche

PAPIER ★

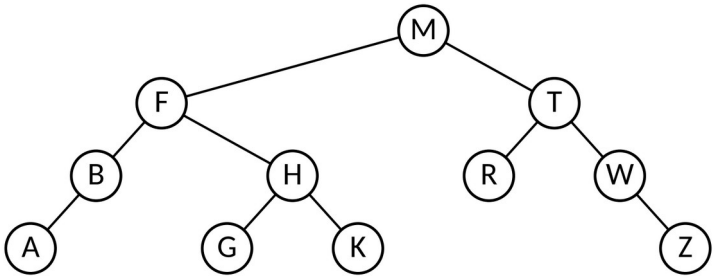


Abbildung 2: Der Baum M (die alte Eiche)

a) Gib die Wurzel, alle Blätter und alle inneren Knoten des Baums M an. **[2 BE]**

b) Gib die Ebene der Knoten T und K, die Höhe von M und die Höhe des Teilbaums mit der Wurzel F an. **[2 BE]**

c) Gib alle Knoten des Teilbaums mit der Wurzel H an. Gib außerdem für jede Ebene an, wie viele Knoten auf ihr liegen. **[1 BE]**

d) Begründe mit der Definition der Höhe, dass ein Blatt die Höhe 1 hat. **[1 BE]**

Infoteil 2: Die Klasse BinTree (Bibliothek „Niedersachsen“, EH)

Operation	Wirkung
c BinTree()	erzeugt einen **leeren** Baum
c BinTree(o: Inhaltstyp)	erzeugt ein **Blatt** mit dem Inhalt o (zwei leere Teilbäume)
isEmpty(): Wahrheitswert	wahr, wenn der Baum leer ist
isLeaf(): Wahrheitswert	wahr, wenn beide Teilbäume leer sind – nur bei nicht leerem Baum
getItem(): Inhaltstyp	liefert den Inhalt der Wurzel – nur bei nicht leerem Baum
setItem(o: Inhaltstyp): nichts	setzt den Inhalt; ein **leerer** Baum wird dadurch zum **Blatt**
getLeft(): BinTree / getRight(): BinTree	liefert den linken bzw. rechten Teilbaum – nur bei nicht leerem Baum
setLeft(b: BinTree) / setRight(b: BinTree)	hängt b als linken bzw. rechten Teilbaum an – nur bei nicht leerem Baum
setEmpty(): nichts	macht den Baum leer

- Ein nicht leerer Baum hat **immer zwei Teilbäume** – notfalls leere Bäume, **nie null**. Die Frage „gibt es ein linkes Kind?“ lautet also `!b.getLeft().isEmpty()`, nicht `b.getLeft() != null`.
- In Signaturen schreibst du den Inhaltstyp dazu: „BinTree vom Inhaltstyp Ganzzahl“ (Zahlenbaum) oder „BinTree vom Inhaltstyp Zeichenkette“ (Termbaum, Monsterquiz).
- getLeft und getRight liefern **Referenzen** auf die echten Teilbäume, keine Kopien: Wer einen Teilbaum verändert, verändert den ganzen Baum.

Casten bei BinTree

BinTree ist **nicht generisch** – getItem() liefert `Object`. Zum Rechnen oder Vergleichen castest du:

```
int x = (Integer) b.getItem();          // BinTree vom Inhaltstyp Ganzzahl
String s = (String) t.getItem();        // BinTree vom Inhaltstyp Zeichenkette
BinTree k = (BinTree) q.dequeue();      // Schlange vom Inhaltstyp BinTree
String text = b.getItem() + " ";        // Verketten geht ohne Cast
```

Abi-Check „Leer?“ – erst fragen, dann zugreifen

getItem, getLeft, getRight, isLeaf, setLeft und setRight auf einem **leeren** Baum führen zum Laufzeitfehler. In rekursiven Methoden ist deshalb **immer der leere Baum der Basisfall**: `if (b.isEmpty()) { … }` steht ganz oben – isLeaf darf erst danach kommen.

Aufgabe 2: Vom Code zum Baum

PAPIER ★

Die folgenden Anweisungen werden nacheinander ausgeführt.

```
1  BinTree w = new BinTree(5);
2  w.setLeft(new BinTree(9));
3  w.setRight(new BinTree(4));
4  w.getLeft().setLeft(new BinTree(2));
5  w.getLeft().getRight().setItem(7);
6  BinTree r = w.getRight();
7  r.getRight().setItem(9);
8  r.getRight().setLeft(new BinTree(1));
```

a) Zeichne den Baum w nach Zeile 8 (leere Teilbäume zeichnest du nicht). **[2 BE]**

b) Gib nach Zeile 8 den Wert der Ausdrücke an. **[2 BE]**

Ausdruck	Wert
`w.getRight().isLeaf()`	
`w.getLeft().getRight().isLeaf()`	
`(Integer) r.getRight().getItem() + 1`	
`w.getRight().getLeft().isEmpty()`	

c) Statt Zeile 6 steht `BinTree r = new BinTree(4);`. Beschreibe, wie die Bäume w und r nach Zeile 8 aussehen, und erläutere den Unterschied zur Originalfassung. **[2 BE]**

d) In der Originalfassung fehlt Zeile 3. Gib an, in welcher Zeile ein Laufzeitfehler auftritt, und begründe. **[1 BE]**

Aufgabe 3: Vom Baum zum Code

PAPIER ★★

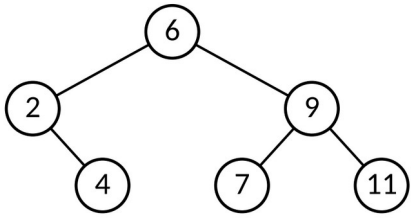


Abbildung 3: Der Baum x

a) Schreibe Anweisungen, die den Baum x aus Abbildung 3 mit Konstruktoren und setLeft/setRight aufbauen. Beginne mit den Teilbäumen. **[2 BE]**

b) Baue x ein zweites Mal auf – diesmal nur mit `BinTree x = new BinTree(6);` und anschließend ausschließlich mit setItem. **[2 BE]**

c) Gib eine Anweisung an, die im fertigen Baum x den Knoten 7 entfernt (er soll ein leerer Teilbaum werden). **[1 BE]**

Infoteil 3: Drei Rundgänge durch einen Baum

Traversierung	Reihenfolge (rekursiv für jeden Teilbaum)	beispielBST (Abbildung 1)
Preorder	Wurzel – links – rechts	`8 3 1 6 4 7 10 14 13`
Inorder	links – Wurzel – rechts	`1 3 4 6 7 8 10 13 14`
Postorder	links – rechts – Wurzel	`1 4 7 6 3 13 14 10 8`

Als Zeichenkette steht hinter **jedem** Element genau ein Leerzeichen; der leere Baum liefert "". Die rekursive Methode ist kurz – sie folgt direkt der Definition:

```
public static String preorder(BinTree b) {
    if (b.isEmpty()) {
        return "";
    }
    return b.getItem() + " " + preorder(b.getLeft()) + preorder(b.getRight());
}
```

Tipp „Rundgang“: Umrunde den Baum außen, links an der Wurzel beginnend, gegen den Uhrzeigersinn. Notierst du jeden Knoten, wenn du **links** an ihm vorbeikommst, entsteht die Preorder; **unten** (zwischen den Kindern) die Inorder; **rechts** die Postorder. Die **Levelorder** (Ebene für Ebene) folgt in Teil 3.

Aufgabe 4: Rundgänge durch die Eiche

PAPIER ★

a) Gib Preorder, Inorder und Postorder des Baums M (Abbildung 2) an. **[3 BE]**

Traversierung	Folge der Knoten
Preorder	
Inorder	
Postorder	

b) Was fällt dir an der Inorder auf? Stelle eine Vermutung auf, welche Eigenschaft des Baums das bewirkt (Ausblick auf DS 34). **[1 BE]**

c) Von einem Baum kennt man nur die Preorder `W A L D H O R N` und die Inorder `A D L H W R O N`. Rekonstruiere den Baum schrittweise, zeichne ihn und gib zur Kontrolle seine Postorder an. Tipp: Das erste Element der Preorder ist die Wurzel – in der Inorder steht links davon der linke, rechts davon der rechte Teilbaum. **[3 BE]**

d) Zeige, dass die Inorder `A B C` allein einen Baum nicht festlegt: Zeichne zwei verschiedene Bäume mit dieser Inorder. Wie viele solcher Bäume gibt es insgesamt? **[2 BE]**

Aufgabe 5: Rechnende Bäume

PAPIER ★★

In einem **Termbaum** (Inhaltstyp Zeichenkette) sind die Blätter Zahlen und die inneren Knoten Rechenzeichen. Ein Rechenzeichen verknüpft die Werte seiner beiden Teilbäume, `/` rechnet ganzzahlig. Der `termbaum()` in Abbildung 1 stellt (3 + 4) * (5 − 2) dar.

a) Zeichne den Termbaum zum Term (9 − 3) / 2 + 4 * 5 (Punkt vor Strich) und gib seinen Wert an. **[2 BE]**

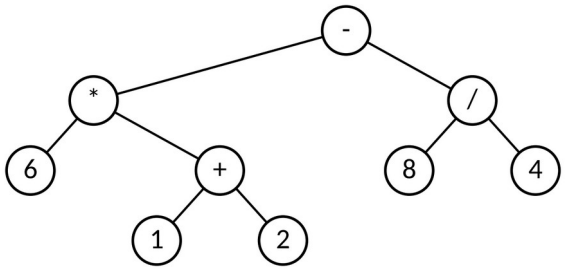


Abbildung 4: Ein Termbaum

b) Gib zum Termbaum aus Abbildung 4 den Term mit allen nötigen Klammern, seinen Wert sowie Preorder und Postorder an. **[3 BE]**

c) Die Postorder eines Termbaums ist die Postfix-Schreibweise aus Welt 4. Zeichne den Termbaum zur Postfix-Folge `5 1 2 + 4 * + 3 -`. Berechne ihren Wert mit einem Stapel: Gib den Stapelinhalt (oben zuerst) nach jedem Zeichen an. **[3 BE]**

Zeichen	5	1	2	+	4	*	+	3	-
Stapel									

d) Erläutere, warum ein Termbaum keine Klammern braucht, die Inorder-Schreibweise aber schon. **[1 BE]**

Aufgabe 6: IDE-Level 5.1 – Bäume ablaufen

IDE ★ – ★★★

Arbeite in der Klasse **Baum** (alle Methoden static), Testklasse `L51BaumTest`. Die Bibliothek „Niedersachsen" ist aus Welt 4 aktiv. Alle Methoden sind **rekursiv** und beginnen mit `if (b.isEmpty())`.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	anzahlKnoten(b: BinTree): Ganzzahl preorder(b: BinTree): Zeichenkette inorder(b), postorder(b): Zeichenkette	`basis_`
★★ Erweiterung	hoehe(b: BinTree): Ganzzahl anzahlBlaetter(b: BinTree): Ganzzahl summe(b: BinTree vom Inhaltstyp Ganzzahl): Ganzzahl	`erweiterung_`
★★★ Experte	zahl(s: Zeichenkette): Ganzzahl (ohne parseInt!) berechne(t: BinTree vom Inhaltstyp Zeichenkette): Ganzzahl spiegeln(b: BinTree): nichts	`experte_`

- Jeder Test ruft deine Methode auch mit dem **leeren Baum** und einem **Blatt** auf. „LAUFZEITFEHLER: … auf einem LEEREN Baum" heißt: Basisfall vergessen oder an der falschen Stelle.
- Die Meldungen zeigen Bäume in Klammerform: `8(3(1,6(4,7)),10(-,14(13,-)))` – Wurzel 8, links 3 (mit 1 und 6), rechts 10 (links leer „-", rechts 14).
- `zahl("907")` liefert 907 – nur mit length() und charAt(): `(int) c - (int) '0'` ist der Wert der Ziffer c. Rechenzeichen vergleichst du mit equals.
- **Baumzeichner** (eigener Workspace, `AQ_W5_Baumzeichner_Geruest.txt`): Aufgaben 1 und 2 – `preorderZeigen` und `postorderZeigen` nach dem Muster von `inorderZeigen`. Die Knoten leuchten in der Besuchsreihenfolge auf – vergleiche mit Aufgabe 4.
- Zum Üben: Warm-up-Karten W5-01 bis W5-04 (BinTree und Casts, Methodenköpfe, Traversierungen, Termbaum).

> 🎯 **Im Lab üben: Level 5.1** „Wegweiser im Wald – Binärbäume und Traversierungen" – Punkte gibt es dort automatisch.

Teil 2 – Rekursiv denken (DS 32)

Infoteil 4: Der Bauplan rekursiver Baummethoden

Fast jede Methode auf Binärbäumen hat denselben Aufbau – nur die Verknüpfung ändert sich:

1. **Basisfall:** Ist der Baum leer, steht das Ergebnis sofort fest (0, "", falsch, −1 …).
2. **Wurzel auswerten:** Inhalt holen (Cast!) und prüfen.
3. **Teilbäume rekursiv:** Dieselbe Methode liefert die Teilergebnisse für links und rechts.
4. **Verknüpfen:** Wurzel und Teilergebnisse zum Ergebnis zusammensetzen.

Beispiel: Wie viele gerade Zahlen enthält der Baum? (zahlenbaum: 2, beispielBST: 5)

```
public static int anzahlGerade(BinTree b) {
    if (b.isEmpty()) {                      // 1. Basisfall: leerer Baum
        return 0;
    }
    int wert = (Integer) b.getItem();       // 2. Wurzel auswerten (Cast!)
    int l = anzahlGerade(b.getLeft());      // 3. Teilbäume rekursiv
    int r = anzahlGerade(b.getRight());
    if (wert % 2 == 0) {                    // 4. verknüpfen
        return 1 + l + r;
    }
    return l + r;
}
```

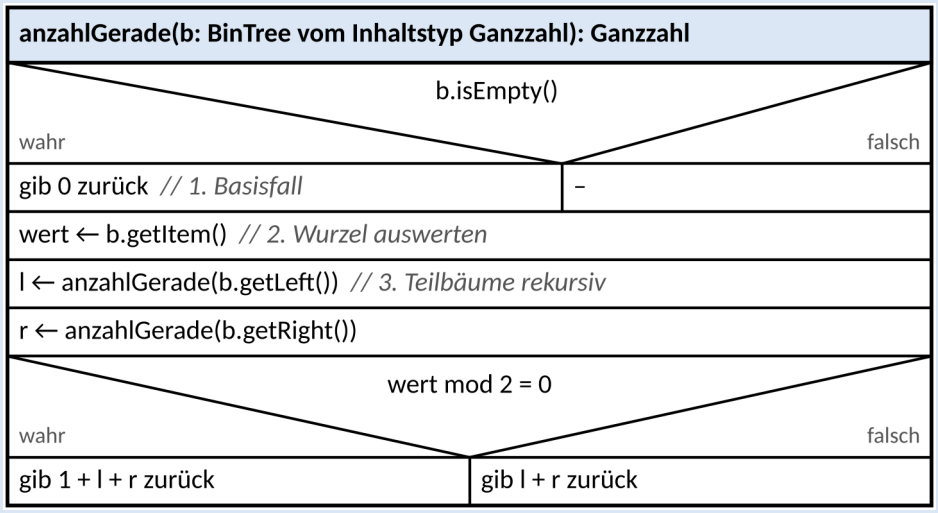


Abbildung 5: Dieselbe Methode als Struktogramm nach EH § 5

Struktogramm-Regeln: Kopf mit Signatur, Zuweisung mit ←, „gib … zurück“ beendet die Operation. Ein rekursiver Aufruf ist eine ganz normale Anweisung: `l ← anzahlGerade(b.getLeft())`. Casts entfallen, Rest bei Division: mod.

Aufgabe 7: Wie hoch reicht der Ast?

PAPIER ★★

Die Methode hoehe wird mit dem Baum t aus Abbildung 6 aufgerufen – dem linken Teilbaum des beispielBST.

```
1  public static int hoehe(BinTree b) {
2      if (b.isEmpty()) {
3          return 0;
4      }
5      int l = hoehe(b.getLeft());
6      int r = hoehe(b.getRight());
7      if (l > r) {
8          return 1 + l;
9      }
10     return 1 + r;
11 }
```

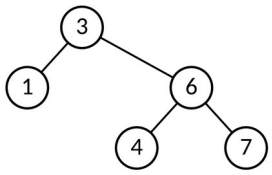


Abbildung 6: Der Baum t

a) Dokumentiere den Aufruf `hoehe(t)` in einer Rekursionstabelle. Trage die Aufrufe in der Reihenfolge ein, in der sie **beginnen**; einen Aufruf mit leerem Teilbaum notierst du als hoehe(leer). In die Spalten l und r gehören die Werte der Variablen l und r. **[4 BE]**

Rekursionstiefe *)	Aufruf	l	r	Rückgabe
0	hoehe(3)			

**) Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Der Erstaufruf hat die Rekursionstiefe 0.*

b) Gib die Anzahl der Aufrufe und die größte Rekursionstiefe an. Bestimme, wie oft hoehe für den ganzen beispielBST aufgerufen wird, und begründe allgemein für einen Baum mit n Knoten. **[2 BE]**

Aufgabe 8: Papier-Compiler im Wald

PAPIER → IDE ★★

Schreibe die Methoden **von Hand** mit vollständigem Java-Methodenkopf (alle static). Gehe danach den Abi-Check durch.

a) Implementiere `anzahlInnere(b: BinTree): Ganzzahl`. Die Methode liefert die Anzahl der inneren Knoten (beispielBST: 5, leerer Baum: 0). **[3 BE]**

b) Implementiere `zaehle(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Ganzzahl`. Die Methode liefert, wie oft x im Baum vorkommt (zahlenbaum: x = 9 liefert 2). Der Baum ist **kein** Suchbaum. **[3 BE]**

c) **Papier-Compiler:** Schreibe `enthaelt(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Wahrheitswert` und `maximum(b: BinTree vom Inhaltstyp Ganzzahl): Ganzzahl` (leerer Baum: −1) aus Level 5.2 (Basis) zuerst vollständig von Hand. Tippe sie dann unverändert ab und starte `L52RekursivTest`. **[4 BE]**

Regeln des Papier-Compilers

1. Schreibe die Methode **vollständig von Hand** – ohne Gerät, ohne Spickzettel außer den Werkzeugkarten.
2. Gehe den Abi-Check durch (Kasten am Ende von Teil 2).
3. Tippe deine Lösung **unverändert** ab – Zeichen für Zeichen, auch mit Fehlern!
4. Starte den Testrunner: Ziel sind grüne Tests `basis_enthaelt` und `basis_maximum` von L52RekursivTest – am besten schon beim ersten Lauf.
5. Jeder Fehler kommt ins Fehlerlogbuch – erst dann korrigierst du in der IDE.

enthaelt:

maximum:

Aufgabe 9: Struktogramme im Unterholz

PAPIER ★★

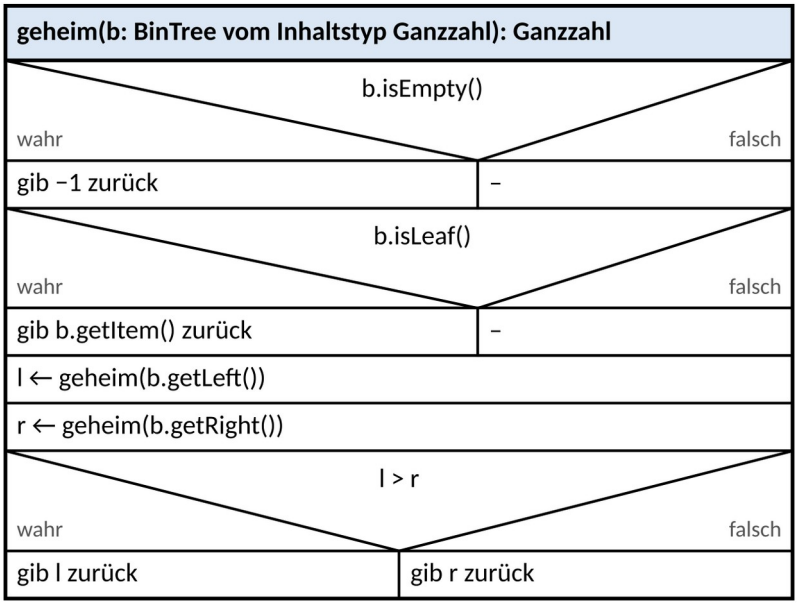
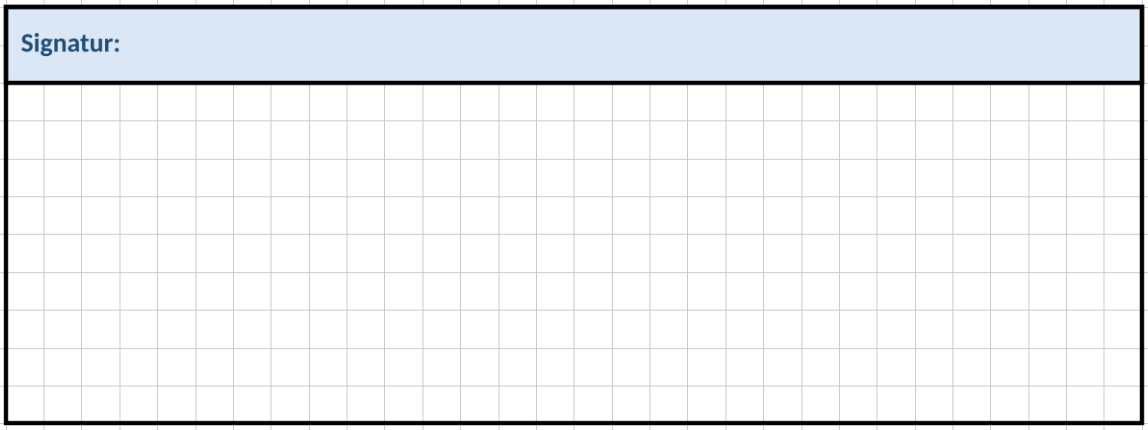


Abbildung 7: Die Operation geheim

a) Bestimme `geheim(b)` für den zahlenbaum und für den beispielBST (Abbildung 1). Gib an, wie oft geheim beim zahlenbaum insgesamt aufgerufen wird. **[3 BE]**

b) Beschreibe, was geheim allgemein berechnet. Gib die Rückgabe für den leeren Baum an und erläutere, warum −1 hier als Ergebnis taugt. **[2 BE]**

c) Entwickle zur Methode zaehle aus Aufgabe 8 b) ein Struktogramm nach den Abi-Konventionen. **[3 BE]**



Aufgabe 10: Fehlerjagd im Unterholz

PAPIER ★★

Die drei Methoden enthalten je einen typischen Fehler.

(A) anzahlKnoten soll die Anzahl der Knoten liefern.

```
1  public static int anzahlKnoten(BinTree b) {
2      if (b.isLeaf()) {
3          return 1;
4      }
5      return 1 + anzahlKnoten(b.getLeft()) + anzahlKnoten(b.getRight());
6  }
```

a) Gib an, was `anzahlKnoten` für den termbaum, den zahlenbaum und den leeren Baum liefert bzw. wo ein Laufzeitfehler entsteht. Korrigiere die Methode. **[3 BE]**

(B) summeGerade soll die Summe aller geraden Werte liefern.

```
1  public static int summeGerade(BinTree b) {
2      if (b.isEmpty()) {
3          return 0;
4      }
5      int w = b.getItem();
6      int s = summeGerade(b.getLeft()) + summeGerade(b.getRight());
7      if (w % 2 == 0) {
8          s = s + w;
9      }
10     return s;
11 }
```

b) Der Compiler meldet einen Fehler. Gib Zeile und Ursache an und korrigiere die Zeile. Gib danach `summeGerade(zahlenbaum)` an. **[2 BE]**

(C) hoehe soll die Höhe liefern (leerer Baum 0, Blatt 1).

```
1  public static int hoehe(BinTree b) {
2      if (b.isEmpty()) {
3          return 0;
4      }
5      int l = hoehe(b.getLeft());
6      int r = hoehe(b.getRight());
7      if (l > r) {
8          return l;
9      }
10     return r + 1;
11 }
```

c) Gib die Rückgaben für den beispielBST und für den entarteten Baum (1, 2, 3, 4, 5 eingefügt) an. Finde und korrigiere den Fehler. Erläutere, warum ein Test, der nur den entarteten Baum prüft, den Fehler nicht findet. **[3 BE]**

Aufgabe 11: IDE-Level 5.2 – Rekursive Baumalgorithmen

IDE ★ – ★★★

Klasse **Rekursiv** (static), Testklasse `L52RekursivTest`. Die Zahlenbäume sind **keine** Suchbäume: Ein Wert kann überall und auch mehrfach stehen.

Stufe	Methoden	Tests beginnen mit
★ Basis	enthaelt(b: BinTree vom Inhaltstyp Ganzzahl, x: Ganzzahl): Wahrheitswert maximum(b): Ganzzahl (leer: −1) anzahlGroesser(b, grenze: Ganzzahl): Ganzzahl	`basis_`
★★ Erweiterung	sammleInorder(b): dynamische Reihung blaetter(b): dynamische Reihung anzahlAufEbene(b, ebene: Ganzzahl): Ganzzahl	`erweiterung_`
★★★ Experte	pfadZu(b, x: Ganzzahl): Zeichenkette ("" Wurzel, "-" fehlt) istGleich(a: BinTree, b: BinTree): Wahrheitswert	`experte_`

- sammleInorder und blaetter erzeugen **eine** DynArray und geben sie an die überladene Hilfsmethode `sammleInorder(b, d)` bzw. `blaetter(b, d)` weiter – alle Aufrufe schreiben in dieselbe Reihung (Warm-up W5-08).
- Rekursionstabellen und Struktogramme trainierst du zusätzlich mit den Warm-up-Karten W5-05 und W5-07.

Abi-Check Binärbäume

- ☐ Basisfall **immer** `if (b.isEmpty())` – ganz oben; isLeaf erst danach.
- ☐ Vor getItem, getLeft, getRight, isLeaf: Ist der Baum sicher nicht leer?
- ☐ Inhalt casten: `(Integer)`, `(String)`, `(BinTree)`; Zeichenketten mit equals vergleichen.
- ☐ Rückgabe in **jedem** Pfad; Ergebnisse der rekursiven Aufrufe verwenden (nicht verwerfen).
- ☐ Rekursionstabelle: Erstaufruf Tiefe 0, Aufrufe in der Reihenfolge ihres Beginns, leere Teilbäume mitzählen.
- ☐ Höhe: leerer Baum 0, Blatt 1; Ebene: Wurzel 0 – oder die Definition aus dem Material.

🎯 **Im Lab üben: Level 5.2** „Rekursion im Geäst – rekursive Baumalgorithmen“ – Punkte gibt es dort automatisch.

Teil 3 – Kür: Der Akinator lernt (DS 33)

Infoteil 5: Ein Entscheidungsbaum, der lernt

Der Akinator errät Monster mit Ja/Nein-Fragen. Sein Wissen steckt in einem **Entscheidungsbaum** (BinTree vom Inhaltstyp Zeichenkette): Innere Knoten sind **Fragen**, Blätter sind **Monster**. Bei „ja“ geht es nach **links**, bei „nein“ nach **rechts**.

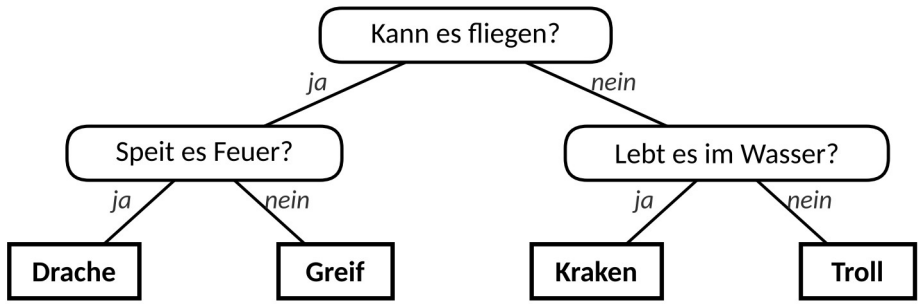


Abbildung 8: Der Startbaum monsterquiz()

- `raten(antworten)` geht von der Wurzel aus je Zeichen einen Schritt ('j' links, sonst rechts), bis ein Blatt erreicht ist, und liefert dessen Namen. Überzählige Antworten werden ignoriert; reichen die Antworten nicht bis zu einem Blatt, wird "?" geliefert.
- `lernen(antworten, neu, frage, jaFuerNeu)`: Das über antworten erreichte Blatt (Monster alt) wird zum Frageknoten frage. Seine Kinder sind neu und alt – neu links, wenn jaFuerNeu wahr ist, sonst rechts.
- `anzahlMonster()` zählt die Blätter, `anzahlFragen()` die inneren Knoten.

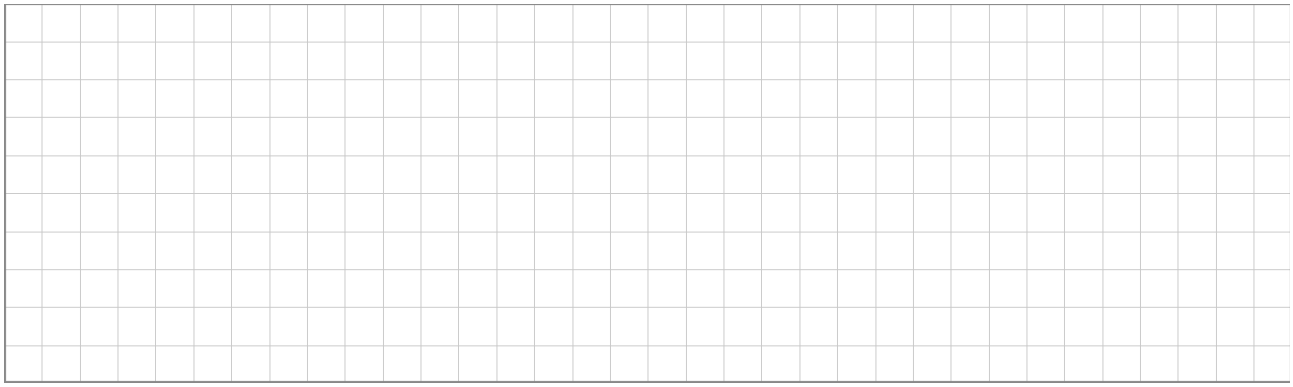
Aufgabe 12: Der Akinator rät und lernt

PAPIER ★★

a) Gib für den Startbaum (Abbildung 8) die Rückgabe von raten an. **[2 BE]**

antworten	"jn"	"nn"	"n"	"nnnj"
raten(antworten)				

b) Nacheinander werden `lernen("jn", "Harpyie", "Hat es ein Menschengesicht?", true)` und `lernen("nj", "Nixe", "Hat es einen Fischschwanz?", true)` aufgerufen. Zeichne den Baum nach beiden Aufrufen. **[3 BE]**



c) Gib für den neuen Baum raten("jnj"), raten("njn") und raten("nj") sowie anzahlMonster() und anzahlFragen() an. **[2 BE]**

d) Begründe, dass in jedem Akinator-Baum, der mit lernen entsteht, gilt: Anzahl der Monster = Anzahl der Fragen + 1. **[2 BE]**

Aufgabe 13: Ebene für Ebene – mit der Schlange

PAPIER ★★

Die **Levelorder** durchläuft einen Baum Ebene für Ebene, jede Ebene von links nach rechts. Rekursion hilft hier nicht – die Methode benutzt eine Schlange vom Inhaltstyp BinTree: In der Schlange stehen **Teilbäume**, keine Zahlen.

```
1  public static String levelorder(BinTree b) {
2      String text = "";
3      Queue q = new Queue();
4      if (!b.isEmpty()) {
5          q.enqueue(b);                              // nur nicht leere Baeume einreihen
6      }   // → Tabellenzeile (Startwerte)
7      while (!q.isEmpty()) {
8          BinTree k = (BinTree) q.dequeue();
9          text = text + k.getItem() + " ";
10         if (!k.getLeft().isEmpty()) {
11             q.enqueue(k.getLeft());
12         }
13         if (!k.getRight().isEmpty()) {
14             q.enqueue(k.getRight());
15         }   // → Tabellenzeile
16     }
17     return text;
18 }
```

a) Führe `levelorder(zahlenbaum)` aus. Notiere an jeder mit „// → Tabellenzeile“ markierten Stelle (vor der Schleife und am Ende jedes Durchlaufs) den entnommenen Knoten, den Wert von text und den Inhalt der Schlange (vorne zuerst; jeden Teilbaum gibst du durch seinen Wurzelwert an). **[4 BE]**

Durchlauf	entnommen	text	Schlange (vorne → hinten)
Start	–	""	5

b) Gib an, wie viele Teilbäume beim zahlenbaum höchstens gleichzeitig in der Schlange stehen. Wie viele sind es höchstens bei einem vollständigen Baum mit 4 Ebenen (15 Knoten)? Begründe. **[2 BE]**

c) Erläutere, warum für die Levelorder eine Schlange gebraucht wird, während Pre-, In- und Postorder mit Rekursion auskommen. **[1 BE]**

Aufgabe 14: IDE-Level 5.3 – Akinator

IDE ★ – ★★★

Objekt-Klasse **Akinator** mit dem Attribut `- baum: BinTree`, Testklasse `L53AkinatorTest`. Der Konstruktor `Akinator(b: BinTree)` und `getBaum()` sind vorgegeben. Die rekursiven Hilfsmethoden bekommen den Teilbaum als Parameter – so geht Rekursion auch in einer Objekt-Klasse.

Stufe	Methoden	Tests beginnen mit
★ Basis	c Akinator(start: Zeichenkette) – ein Blatt raten(antworten: Zeichenkette): Zeichenkette	`basis_`
★★ Erweiterung	lernen(antworten, neu, frage: Zeichenkette, jaFuerNeu: Wahrheitswert): nichts anzahlMonster(): Ganzzahl · anzahlFragen(): Ganzzahl	`erweiterung_`
★★★ Experte	levelorder(b: BinTree): Zeichenkette (static, mit Schlange)	`experte_`

- Die restlichen Antworten nach dem ersten Zeichen: `antworten.substring(1, antworten.length())` – Zeichen vergleichst du mit `antworten.charAt(0) == 'j'`.
- **Baumzeichner** Aufgabe 3: `levelorderZeigen` – färbst du eingereihte Knoten mit `Zeichner.betrete` (blau) und ausgegebene mit `Zeichner.besuche` (grün), siehst du, was gerade in der Schlange steht. Vergleiche mit deiner Tabelle aus Aufgabe 13 und mit Warm-up W5-09.

🎯 **Im Lab üben: Level 5.3** „Kür: Der Akinator lernt – Entscheidungsbaum und Level-Order“ – Punkte gibt es dort automatisch.

Für Schnelle: Der klügste Akinator

- **Team-Wettbewerb:** Bringt eurem Akinator im Hauptprogramm möglichst viele Monster mit sinnvollen Fragen bei. Wer hat nach 15 Minuten die meisten Monster – und wie hoch ist euer Baum dann?
- Ein Akinator mit 16 Monstern: Wie viele Fragen braucht er mindestens bzw. höchstens, um jedes Monster zu erraten? Wie müsste der Baum dafür aussehen?
- Experte: `spiegeln` (Level 5.1) auf das Monsterquiz anwenden – was bedeuten danach „ja“ und „nein“?